

Two-hidden-layer extreme learning machine for regression and classification

B.Y. Qu^{a,b}, B.F. Lang^a, J.J. Liang^{a,*}, A.K. Qin^c, O.D. Crisalle^a

^a School of Electrical Engineering, Zhengzhou University, Zhengzhou 450001, China

^b School of Electric and Information Engineering, Zhongyuan University of Technology, Zhengzhou 450007, China

^c School of Computer Science and Information Technology, RMIT University, Melbourne, 3001 Victoria, Australia

ARTICLE INFO

Article history:

Received 28 May 2015

Received in revised form

9 November 2015

Accepted 9 November 2015

Available online 17 November 2015

Keywords:

Extreme learning machine

Two-hidden-layer

Regression

Classification

Neural network

ABSTRACT

As a single-hidden-layer feedforward neural network, an extreme learning machine (ELM) randomizes the weights between the input layer and the hidden layer as well as the bias of hidden neurons, and analytically determines the weights between the hidden layer and the output layer using the least-squares method. This paper proposes a two-hidden-layer ELM (denoted TELM) by introducing a novel method for obtaining the parameters of the second hidden layer (connection weights between the first and second hidden layer and the bias of the second hidden layer), hence bringing the actual hidden layer output closer to the expected hidden layer output in the two-hidden-layer feedforward network. Simultaneously, the TELM method inherits the randomness of the ELM technique for the first hidden layer (connection weights between the input weights and the first hidden layer and the bias of the first hidden layer). Experiments on several regression problems and some popular classification datasets demonstrate that the proposed TELM can consistently outperform the original ELM, as well as some existing multilayer ELM variants, in terms of average accuracy and the number of hidden neurons.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

Single-hidden-layer feedforward neural networks (SLFNs), one of the most popular neural network models [1,2], have a simple structure consisting of one input layer, one hidden layer, and one output layer. A wide range of applications have been used to demonstrate the efficacy of SLFNs [3,4]. However, these techniques suffer from a time-expensive training process that usually adopts gradient-based error back-propagation algorithms, and consequently is prone to getting stuck in local minima. To address this issue, in 2004 Huang et al. [3] proposed an extreme learning machine (ELM) technique aiming at reducing the computational costs incurred by the error back-propagation procedure during the training process. A distinguishing feature of ELMs is that both the connection weights from the input layer to the hidden layer and the hidden neurons' biases are randomly generated, instead of being iteratively learned as in conventional SLFNs. Moreover, the connection weights from the hidden layer to the output layer are analytically determined using the time-efficient least-squares method (LS) [5]. As a result, an ELM features remarkably fast

training speed and outstanding generalization performance. The ELM approach has demonstrated its advantages in various fields of applications, including image recognition [6–10], power-load forecasting [11,12], wind speed forecasting [13], and protein structure prediction [14], among others. However, because of the random weights from the input layer to the hidden layer, as well as the random biases of the hidden neurons, the average accuracy of ELM variants is generally low, which calls for further investigation of better hidden-layer parameter calculation approaches.

Many ELM variants have been developed to improve specific aspects of the performance of the original algorithm. Examples include voting-based extreme learning machines (V-ELM) [15], regularized extreme learning machines (RELM) [16,17], evolutionary extreme learning machines (E-ELM) [18], online sequential extreme learning machines (OS-ELM) [19], fully complex extreme learning machines (Fully complex ELM) [4,20], sparse extreme learning machines (Sparse ELM) [21], kernel-based extreme learning machines [22], and pruned-extreme learning machines (P-ELM) [23], among others. However, the problem of how to achieve more satisfactory accuracy remains a challenge to overcome.

To achieve desirable accuracy improvements, we propose a two-hidden-layer extreme learning machine (TELM) algorithm, which adds a hidden layer to the single-hidden-layer ELM architecture, and utilizes a novel method to calculate the parameters

* Corresponding author. Tel.: +86 13526781788.

E-mail addresses: qby1984@hotmail.com (B.Y. Qu), langbofei@hotmail.com (B.F. Lang), liangjing@zzu.edu.cn (J.J. Liang), kai.qin@rmit.edu.au (A.K. Qin), crisalle@gmail.com (O.D. Crisalle).

related to the second hidden layer (namely, connection weights between the first and second hidden layer and the bias of the second hidden layer). Based on previous research, two-hidden-layer feedforward neural networks (TLFNs) [24] typically require fewer hidden neurons than SLFNs to achieve a desired performance level. This is an initial basis for considering the two-hidden-layer structure proposed. The foundational ideas for the TELM algorithm are simpler to present by comparing and contrasting its features with other multilayer ELM algorithms.

First consider the hierarchical extreme learning machine (HELM) approach presented in [25], which is based on a hierarchical feedforward neural network (HFNN) structure consisting of two parts, where each part is comprised of one input layer, one hidden layer, and one output. It is therefore possible to regard the output of the first part as an input neuron in the second part. Unlike HELM, the proposed TELM contains only one output layer, and is specifically designed for training the parameters of the hidden layers. Furthermore, HELM is tailored to solving real-time or on-line prediction problems that involve a time-sequence dataset (such as predicting the water quality in a wastewater treatment processes, for example), whereas TELM as no such restriction on the type of training dataset.

Next, consider the multilayer extreme learning machine (ML-ELM) [26] and the alternative H-ELM advanced in [27]. Both techniques involve ELM-based auto-encoder schemes as their building blocks. In fact, this H-ELM method is an improvement over ML-ELM, as it features a sparse ELM auto-encoder for improved performance. Both schemes focus mainly on solving classification problems, as they are involved in feature extraction. In their mode of operation, previous hidden layers specialize on processing for feature extraction, whereas the last hidden layers are mostly intended for least-squares operations. The focus of the proposed TELM is different, as it seeks to obtain improved performance using a reduced number of hidden neurons. However, the TELM can also incorporate ELM-based auto-encoder techniques, hence making it a suitable alternative for seeking improved performance in feature extraction problems under scenarios that call for a reduced number of neurons.

The experimental results presented in this paper for several regression and classification problems demonstrate the superiority of TELM over the original ELM and also over other multilayer ELM variants in terms of average accuracy. Our experiments also investigate the different effect on regression and classification problems observed when using initial orthogonalization procedures applied to the parameters of the first hidden-layer (that is, connection weights between the input weights and the first hidden layer and the bias of the first hidden layer).

The rest of this paper is organized as follows: Section 2 presents a brief review of the original ELM, Section 3 describes the proposed TELM technique, Section 4 reports and analyzes experimental results, and finally, Section 5 draws key conclusions and also discusses future research plans.

2. Extreme learning machine

The ELM approach originally proposed by Huang et al. [3] aims at avoiding a time-consuming iterative training procedure and simultaneously improving the generalization performance. The idea is inspired by the biological thought that the human brain is a sophisticated system that can handle diverse tasks, day and night, without human intervention. Based on this reasoning, some researchers strongly support the idea that there must be some parts of the brain where the neuron configurations do not depend on the external environment [3,24,28–30]. The ELM algorithm takes advantage of this biological argument, and employs tuning-free

neurons in the hidden layer to resolve the adverse issues encountered by the back-propagation [31] and Levenberg–Marquardt algorithms [32].

Consider N arbitrary distinct samples $(\mathbf{x}_i, \mathbf{t}_i) (i = 1, 2, \dots, N)$, i.e., there is an input feature $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T$ and a desired matrix $\mathbf{T} = [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_N]^T$ comprised of labeled samples, where $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{in}]^T \in \mathbb{R}^n$ and $\mathbf{t}_i = [t_{i1}, t_{i2}, \dots, t_{im}]^T \in \mathbb{R}^m$, where the superscript “ T ” denotes the matrix/vector transposition. Let L denote the number of hidden neurons with activation function $g(\mathbf{x})$. The ELM method selects in a random way the input-weight matrix $\mathbf{W} = [\mathbf{W}_1, \mathbf{W}_2, \dots, \mathbf{W}_j]^T \in \mathbb{R}^{L \times n}$ that links the input layer to the hidden layer, and the bias vector $\mathbf{B} = [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_L]^T \in \mathbb{R}^{L \times n}$ of the hidden-layer neurons. Furthermore, $\mathbf{W}$ and $\mathbf{B}$ are determined simultaneously, and they remain fixed during the training phase. This procedure allows transforming the original nonlinear neural-network system to a system described by the linear expression

$$\mathbf{H}\boldsymbol{\beta} = \mathbf{T} \quad (1)$$

where $\boldsymbol{\beta} = [\boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \dots, \boldsymbol{\beta}_L]^T \in \mathbb{R}^{L \times m}$ is the connection-weight matrix between the hidden layer and the output layer, with vector components $\boldsymbol{\beta}_j = [\beta_{j1}, \beta_{j2}, \dots, \beta_{jm}]^T (j = 1, 2, \dots, L)$ that denote the connection weights between the j th hidden neuron and m output neurons, $\mathbf{H} = g(\mathbf{W}\mathbf{X} + \mathbf{B}) \in \mathbb{R}^{N \times L}$ is the hidden layer output matrix whose scalar entries $h_{ij} = g(\mathbf{W}_j\mathbf{x}_i + \mathbf{b}_j) (i = 1, 2, \dots, N, j = 1, 2, \dots, L)$ are interpreted as the output of the j th hidden neuron with respect to $\mathbf{x}_i$, $\mathbf{W}_j = [W_{j1}, W_{j2}, \dots, W_{jn}]^T$ is the vector of connection weights between n input neurons and the j th hidden neuron, and where $\mathbf{b}_j$ is the bias of the j th hidden neuron. Finally, the matrix-vector product $\mathbf{W}_j\mathbf{x}_i$ is interpreted as the inner product between matrix $\mathbf{W}_j$ and vector $\mathbf{x}_i$.

The only parameter to be calculated in the ELM is the output-weights matrix $\boldsymbol{\beta}$. Using the least-squares method it follows that

$$\boldsymbol{\beta} = \mathbf{H}^\dagger \mathbf{T} \quad (2)$$

where $\mathbf{H}^\dagger$ is the Moore–Penrose (MP) generalized inverse of matrix $\mathbf{H}$, which can be calculated using the orthogonal projection method. That is to say, if $\mathbf{H}^T \mathbf{H}$ is nonsingular, then $\mathbf{H}^\dagger = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$; otherwise $\mathbf{H}^\dagger = \mathbf{H}^T (\mathbf{H} \mathbf{H}^T)^{-1}$ when $\mathbf{H} \mathbf{H}^T$ is nonsingular. A benefit of using the MP method of solution is that the above formula yields the solution vector $\boldsymbol{\beta}$ of the least two-norm when $\mathbf{H} \mathbf{H}^T$ is nonsingular, a valuable advantage when recognizing that Bartlett [33] observes that smaller weights lead to improved generalization performance.

The implementation of the original ELM proceeds according to the following steps, given N training samples $(\mathbf{x}_i, \mathbf{t}_i) (i = 1, 2, \dots, N)$ and L hidden neurons with activation function $g(\mathbf{x})$:

- (i) Randomly assign the connection weights between the input layer and the hidden layer $\mathbf{W}$ and the bias of the hidden layer $\mathbf{B}$.
- (ii) Calculate the hidden layer output matrix $\mathbf{H} = g(\mathbf{W}\mathbf{X} + \mathbf{B})$.
- (iii) Obtain weights between the hidden layer and the output layer using the least-square method $\boldsymbol{\beta} = \mathbf{H}^\dagger \mathbf{T}$.

3. Two-hidden-layer extreme learning machine

In 1997 Tamura and Tateishi [34] demonstrated that two-hidden-layer feedforward networks (TLFNs) are superior to SLFNs in terms of the ability to use fewer hidden neurons to achieve the desired performance. They claimed that a TLFN with only $(N/2 + 3)$ hidden neurons can learn from N training samples to achieve any negligible training error. Huang [24] further demonstrates that by using $2\sqrt{(m+3)N}$ hidden neurons a TLFN can learn from N training samples to achieve an arbitrarily small training error. Such advantage of TLFNs motivates us to translate

the ideas behind ELM into a TLFN framework. Thus our proposed algorithm is named *Two-Hidden-Layer Extreme Learning Machine* (TELM). The TELM network structure is illustrated in Fig. 1.

The workflow of the TELM architecture is depicted in Fig. 2. Given a set of N training samples $(\mathbf{x}_i, \mathbf{t}_i)$ and $2L$ hidden neurons in total (that is, each of the two hidden layer has L hidden neurons) with the activation function $g(\mathbf{x})$, we first randomly initialize the connection weight matrix between the input layer and the first hidden layer $\mathbf{W}$ and the bias matrix of the first hidden layer $\mathbf{B}$, and then calculate the weight matrix β between the second hidden layer and the output layer using Eq. (2). According to the workflow of Fig. 2, it follows that

$$g(\mathbf{W}_H \mathbf{H} + \mathbf{B}_1) = \mathbf{H}_1 \quad (3)$$

where $\mathbf{W}_H$ denotes the weight matrix between the first hidden layer and the second hidden layer. We assume that the first and second hidden layers have the same number of neurons, and thus $\mathbf{W}_H$ is a square matrix. The notation $\mathbf{H}$ denotes the output between the first hidden layer with respect to all N training samples. The matrices $\mathbf{B}_1$ and $\mathbf{H}_1$ respectively represent the bias and the expected output of the second hidden layer.

The expected output of the second hidden layer can be calculated as

$$\mathbf{H}_1 = \mathbf{T} \beta^\dagger \quad (4)$$

where $\beta^\dagger$ is the MP generalized inverse of the matrix β . The calculating method of $\beta^\dagger$ is the same as previously discussed for $\mathbf{H}^\dagger$, namely $\beta^\dagger = (\beta^T \beta)^{-1} \beta^T$ if $\beta^T \beta$ is nonsingular, or alternatively

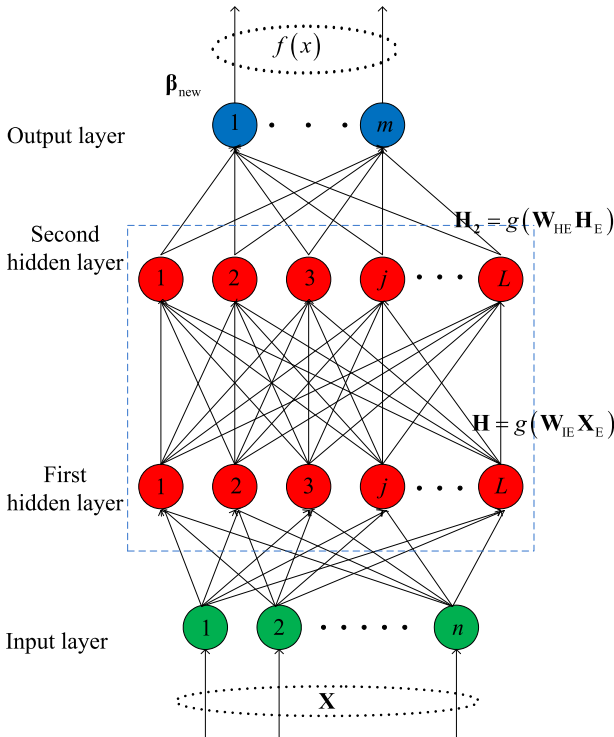


Fig. 1. Structure of the proposed TELM approach.

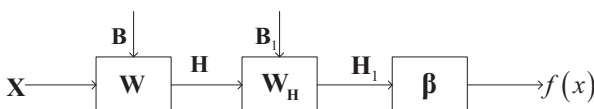


Fig. 2. Workflow of the proposed TELM approach.

$\beta^\dagger = \beta^T (\beta^T \beta)^{-1}$ if $\beta^T \beta$ is nonsingular. Subsequently we define the augmented matrix $\mathbf{W}_{HE} = [\mathbf{B}_1 \ \mathbf{W}_H]$, and calculate it as

$$\mathbf{W}_{HE} = g^{-1}(\mathbf{H}_1) \mathbf{H}_E^\dagger \quad (5)$$

where $\mathbf{H}_E^\dagger$ is the MP generalized inverse of $\mathbf{H}_E = [\mathbf{1} \ \mathbf{H}]^T$, $\mathbf{1}$ denotes a one-column vector of size N whose elements are the scalar unit 1, where the notation $g^{-1}(\mathbf{x})$ indicates the inverse of the activation function $g(\mathbf{x})$. The calculation of $\mathbf{H}_E^\dagger$ proceeds in the fashion described before.

The experiments conducted to test the performance of the proposed TELM algorithm involve different activation functions for regression and classification cases. For classification purposes we adopt the widely used logistic sigmoid function $g(\mathbf{x}) = 1/(1 + e^{-\mathbf{x}})$. On the other hand, for regression problems we invoke the hyperbolic tangent function $g(\mathbf{x}) = (1 - e^{-\mathbf{x}})/(1 + e^{-\mathbf{x}})$, which is a simple translation and scaling of the logistic sigmoid function. We prefer utilizing the hyperbolic tangent function in regression analysis because it yields an output distribution that is symmetrical on both sides of zero, leading to enhanced stability for solving regression problems. The actual output of the second hidden layer is calculated as

$$\mathbf{H}_2 = g(\mathbf{W}_{HE} \mathbf{H}_E) \quad (6)$$

and finally, the weight matrix β_{new} between the second hidden layer and the output layer is calculated as

$$\beta_{\text{new}} = \mathbf{H}_2^\dagger \mathbf{T} \quad (7)$$

where $\mathbf{H}_2^\dagger$ is the MP generalized inverse of $\mathbf{H}_2$, obtained using the approach discussed before. The TELM output after training can be expressed as

$$f(\mathbf{x}) = \mathbf{H}_2 \beta_{\text{new}} \quad (8)$$

To make the final actual hidden output approach the expected hidden output, during the training phase the TELM adds an innovative parameter-setting step for the second hidden layer of TLFN, as described in Algorithm 1.

In addition to the above description, it is important to point out regarding Eq. (5) that it is necessary to take appropriate precautions to guarantee the feasibility of the inversion of the expected output of the second layer. This is accomplished by recognizing that when one calculates the second hidden layer parameters (connection weights between the first and second hidden layer and the bias of the second hidden layer), $\mathbf{H}_1$ needs to be normalized in the range between -0.9 and 0.9 whenever the maximum of $\mathbf{H}_1$ is greater than 1 or the minimum of $\mathbf{H}_1$ is less than -1 . Of course, $\mathbf{H}_2$ must then be denormalized accordingly.

Remark 1. It is worth noting that an orthogonal initialization is added to the first step of all the algorithms involved in the experiments on classification datasets, because it is observed that this type of initialization yields better performance for classification problems. In contrast, a random initialization performs better on regression problems. A simple experiment is presented in the following section to support this claim.

Algorithm 1. TELM Algorithm

Input: N training samples $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T$,

$\mathbf{T} = [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_N]^T$, and $2L$ hidden neurons in total with activation function $g(\mathbf{x})$

- 1: Randomly generate the connection weight matrix between the input layer and the first hidden layer $\mathbf{W}$ and the bias matrix of the first hidden layer $\mathbf{B}$ and for simplicity, $\mathbf{W}_{IE}$ is defined as $[\mathbf{B} \ \mathbf{W}]$ and similarly, $\mathbf{X}_E$ is defined as $[\mathbf{1} \ \mathbf{X}]^T$.
- 2: Calculate $\mathbf{H} = g(\mathbf{W}_{IE} \mathbf{X}_E)$
- 3: Obtain weight matrix between the second hidden layer and the output layer $\beta = \mathbf{H}^\dagger \mathbf{T}$

4: Calculate the expected output of the second hidden layer

$$\mathbf{H}_1 = \mathbf{T} \boldsymbol{\beta}^\dagger$$

5: Determine the parameters of the second hidden layer (connection weight matrix between the first and second hidden layer and the bias of the second hidden layer)

$$\mathbf{W}_{\text{HE}} = \mathbf{g}^{-1}(\mathbf{H}_1) \mathbf{H}_E^\dagger$$

6: Obtain the actual output of the second hidden layer

$$\mathbf{H}_2 = \mathbf{g}(\mathbf{W}_{\text{HE}} \mathbf{H}_E)$$

7: Recalculate the weight matrix between the second hidden layer and the output layer $\boldsymbol{\beta}_{\text{new}} = \mathbf{H}_2^\dagger \mathbf{T}$

Output: The final output of TELM is

$$f(x) = \mathbf{g}\{[\mathbf{W}_{\text{HG}}(\mathbf{W} \mathbf{X} + \mathbf{B}) + \mathbf{B}_1]\} \boldsymbol{\beta}_{\text{new}}$$

4. Performance evaluation

To test the performance of the proposed TELM, our experiments are divided into three parts: regression problems, simple benchmark classification datasets, and a more complex classification problem based on the MNIST dataset. All the experiments are conducted in the MATLAB R2013b computational environment running on a computer with a 2.53 GHz i3 CPU. Furthermore, to comprehensively compare resulting performances, each algorithm used in the experiments is uniformly assigned a number of hidden neurons varying from 100 to 500. The number of hidden neurons is increased in steps of 20 until reaching the total number 500. Moreover, 20 trials are carried out for each algorithm.

4.1. Regression

To solve time-consuming complex optimization problems, researchers currently prefer incorporating surrogate models in the optimization algorithms. A fast regression algorithm with good estimation accuracy provides a desirable choice. The following three widely used optimization functions [35] are used in this subsection to generate the training and testing data for evaluating the performance of the algorithms under consideration:

- 1) $f_1(x) = \sum_{i=1}^D x_i^2$
- 2) $f_2(x) = 20 - 20e^{-0.2\sqrt{\frac{1}{n}\sum_{i=1}^n x_i^2/D}} - e^{\sum_{i=1}^n \cos(2\pi x_i)/D} + e$
- 3) $f_3(x) = \sum_{i=1}^D [x_i^2 - 10\cos(2\pi x_i) + 10]$

where the symbol D denotes the dimension of the function. Here $f_1(x)$ is a simple unimodal nonlinear function, while $f_2(x) \sim f_3(x)$ are complex multimodal nonlinear functions. Each problem involves 1000 training samples and 1000 testing samples. Moreover, the dimension of each function is assigned as $D = 10$. In all function-approximation experiments, the 10 input attributes are normalized to the range $[0, 1]$ and the output attributes are normalized to the range $[-1, 1]$. The performance evaluation criteria selected for the comparative study is the average accuracy quantified in terms of the root mean square error (RMSE) for the regression problems.

In this subsection, we first conduct a series of experiments to determine which kind of initialized parameters technique can make the algorithms perform better between the random initialization and the orthogonal initialization on regression problems. For succinctness of exposition, we report in Fig. 3 only the case for the original ELM. It can be readily concluded from the figure that the orthogonal initialization is not suitable for use in regression problems.

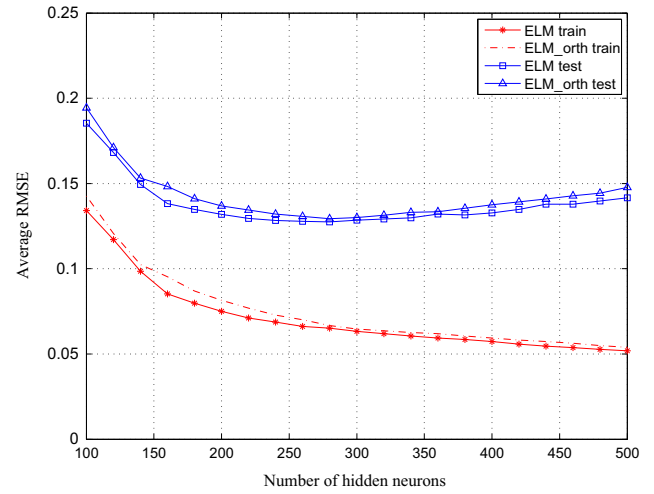


Fig. 3. Performance comparison between the original ELM and ELM with orthogonal initialization on $f_2(x)$.

Fig. 4 shows the average RMSE for regression for the algorithms considered, namely ELM, TELM, and an algorithmic variant that we denote TELM_rand in which the parameters of the two hidden layers (connection weights between the input layer and the first hidden layer, the bias of the first hidden layer, connection weights between the first and second hidden layer and the bias of the second hidden layer) are generated randomly as in the case of the original ELM. Small RMSE values indicate a better accuracy of regression. From Fig. 4 it can be concluded that both the average training RMSE and the average testing RMSE of the TELM algorithm are dramatically superior to those of the ELM and TELM_rand algorithms when the number of hidden neurons ranges from 100 to 300. It is therefore inferred that the proposed TELM algorithm reaches a superior performance under conditions where there is a relatively small number of hidden neurons.

From the time-cost aspect, the average training speed for each of the three algorithms considered is extremely fast, and of a similar order of magnitude. Further details of the training time are not reported here due to space limitations.

4.2. Simple benchmark classification datasets

According to previous literature [26], the orthogonalization of randomly initialized connection weights between the input layer and the first hidden layer and biases of the first hidden layer (parameters of the first hidden layer) tends to bring about better generalization performance on classification applications. Therefore, the random parameters of the first hidden layer are required to be orthogonal in all classification simulations reported in this work.

To test the performance of the proposed algorithm on simple benchmark classification datasets, five commonly used datasets, denoted vowel, satellite, pendigits, optdigits and segment, are collected from the *Machine Learning Repository* of the University of California, Irvine [36]. Specifications for these five datasets are given in Table 1. For each case, the training and testing datasets are randomly generated from their corresponding overall datasets.

The classification error percentage of the testing data is chosen as the performance evaluation criteria for these classification problems. Fig. 5 shows the experimental results plotted on logarithmic axes, reporting the performance for the three algorithms investigated, namely TELM, ELM, and TELM_rand. As in the case of the regression problems, the training speed of all three algorithms considered is very fast, with differences smaller than one order of magnitude.

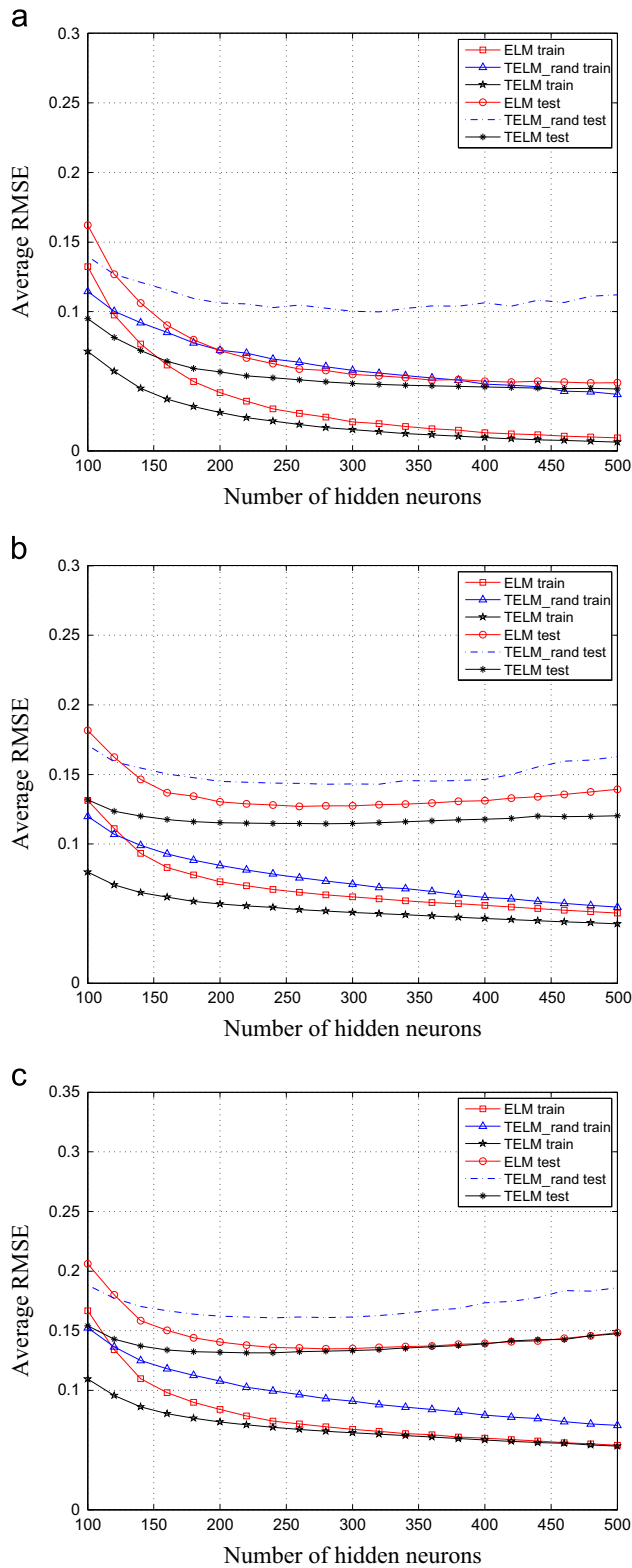


Fig. 4. Average RMSE for the algorithms TELM, ELM, and TELM_rand using three different functions. (a) Case of the simple unimodal function $f_1(x)$, (b) case of the complex unimodal function $f_2(x)$, and (c) case of the complex unimodal function $f_3(x)$.

A smaller classification error percentage indicates a better classification-problem performance. Fig. 5 shows that the proposed TELM algorithm achieves a lower testing classification error percentage among all benchmark datasets, relative to the ELM and TELM_rand techniques, when the number of hidden neurons is

Table 1

Specifications for the classification datasets.

Datasets	Training samples	Testing samples	Attributes	Classes
Vowel	660	330	14	11
Satellite	4400	2000	35	7
Pendigits	7494	3498	16	10
Optdigits	3823	1797	64	10
Segment	1500	810	18	7

below 200. However, for a number higher than 200 neurons, the performance of TELM is similar or at worst only slightly inferior to that of ELM or TELM_rand for the segment dataset case.

To investigate why the proposed TELM algorithm specified with fewer hidden neurons can achieve smaller testing error percentages in classification applications, it is useful to analyze the ratio of the average interclass distance and the average intraclass distance of $\mathbf{H}_i$ for all three algorithms. More specifically, let D_{intra} denote the intraclass distance and D_{inter} represent the average interclass distance, and then define the *average class distance ratio*

$$D_{\text{avg}} = D_{\text{intra}}/D_{\text{inter}}$$

where

$$D_{\text{intra}} = \sum_{i=1}^{C_{\text{num}}} \text{mean} \left\{ \text{mean} \left[\max(\mathbf{H}_i^T) - \min(\mathbf{H}_i^T) \right]^2 \right\}$$

$$D_{\text{inter}} = \sum_{i=1}^{C_{\text{num}}} \sum_{j>i}^{C_{\text{num}}} \text{mean} \left\{ \left[\text{mean}(\mathbf{H}_i^T) - \text{mean}(\mathbf{H}_j^T) \right]^2 \right\}$$

and where the symbol $\mathbf{H}_i$ is the final hidden layer output matrix that belongs to the i th class for each algorithm, $\mathbf{H}_j$ is the final hidden layer output matrix that belongs to the j th class, and C_{num} represents the number of classes in one dataset. The above formulas reveal that the average intraclass distance denotes the average distance of samples in the same target matrix group, while the average interclass distance represents the average distance of samples among all pairs of target matrix groups. Smaller values of D_{avg} indicate smaller average error percentage for the classification problem, as this implies a smaller average intraclass distance and a larger average interclass distance.

The analysis now focuses on the *vowel* datasets as an example. The results are reported in Fig. 6, which presents the average class distance ratio D_{avg} for the training data in Fig. 6(a), and for the testing data in Fig. 6(b). From this figure it is readily concluded that for both cases of the training dataset and testing dataset, no matter how much the number of hidden neurons grows the average class distance ratio for the TELM technique is always evidently smaller than that of the two other algorithms considered. The implications of these observations are that TELM technique is a preferred alternative in terms of improvement of average testing accuracy when the total number of hidden neurons that can be deployed is reduced, such as occurs in applications where there is a shortage of computational storage devices.

Remark 2. When comparing the TELM technique with the original ELM, it is apparent that the proposed TELM approach adds another hidden layer into the SLFN structure, while the parameters of the second hidden layer (connection weights between the first and second hidden layer and the bias of the second hidden layer) are set by the new setting technique. In this way, the TELM algorithm injects input features into a more complex mapping relationship, and hence involves more calculations. This leads to the generation of a TELM output that is more accurate than the output of the ELM technique. Note that the most salient contrast between the

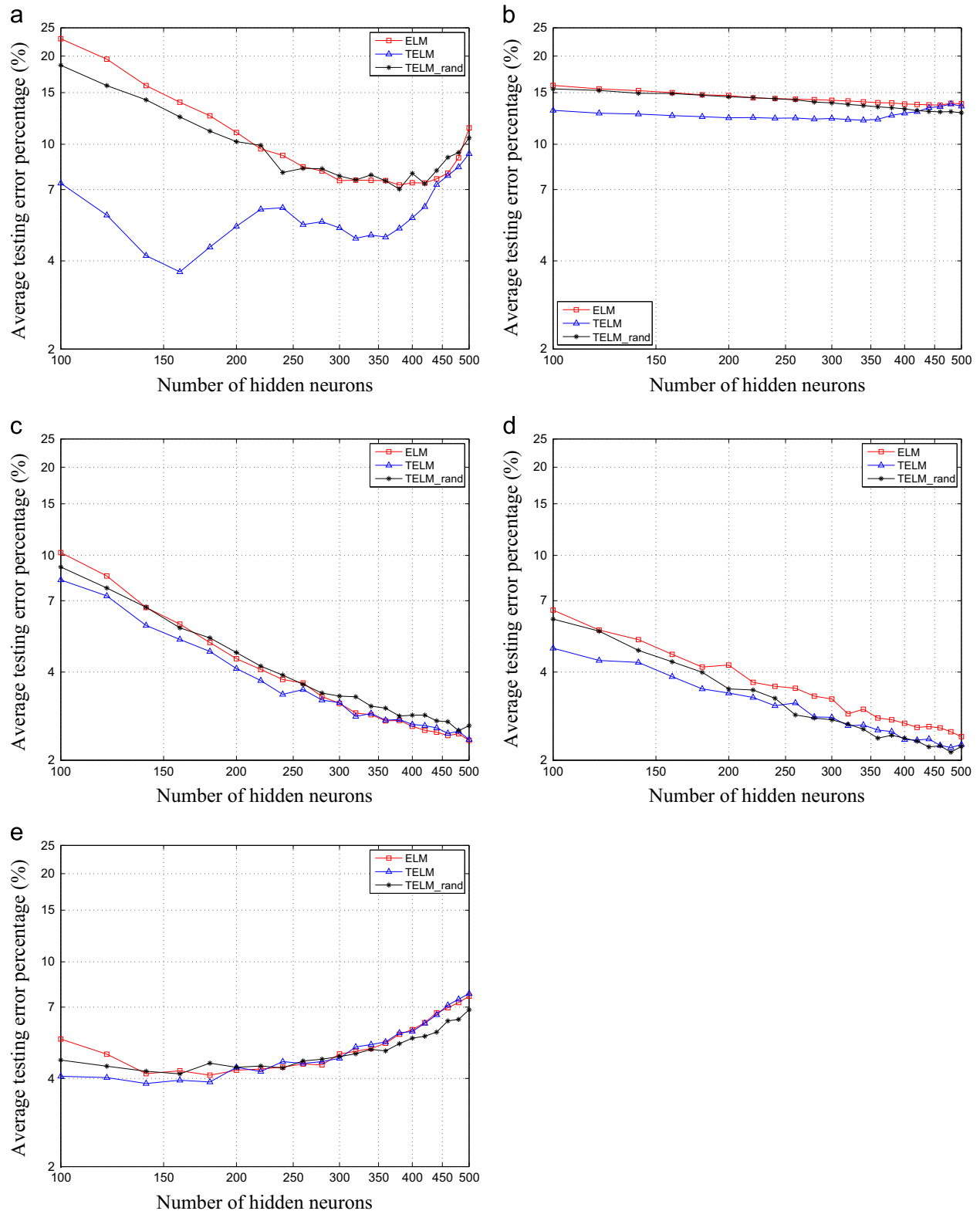


Fig. 5. Average testing classification error percentage for the algorithms TELM, ELM, and TELM_rand using (a) vowel, (b) satellite, (c) pendigits, (d) optdigits, and (e) segment dataset.

proposed TELM and TELM_rand is that the former includes an explicit method of calculating the parameters for the second hidden layer. Instead of randomly generated parameters used in both hidden layers of TELM_rand, the key idea behind TELM is its

deliberate focus on trying to make the actual hidden layer output as close as possible to the expected hidden layer output. As a result, it is easy to find a better mapping relationship between input and output signals, and as a consequence the TELM

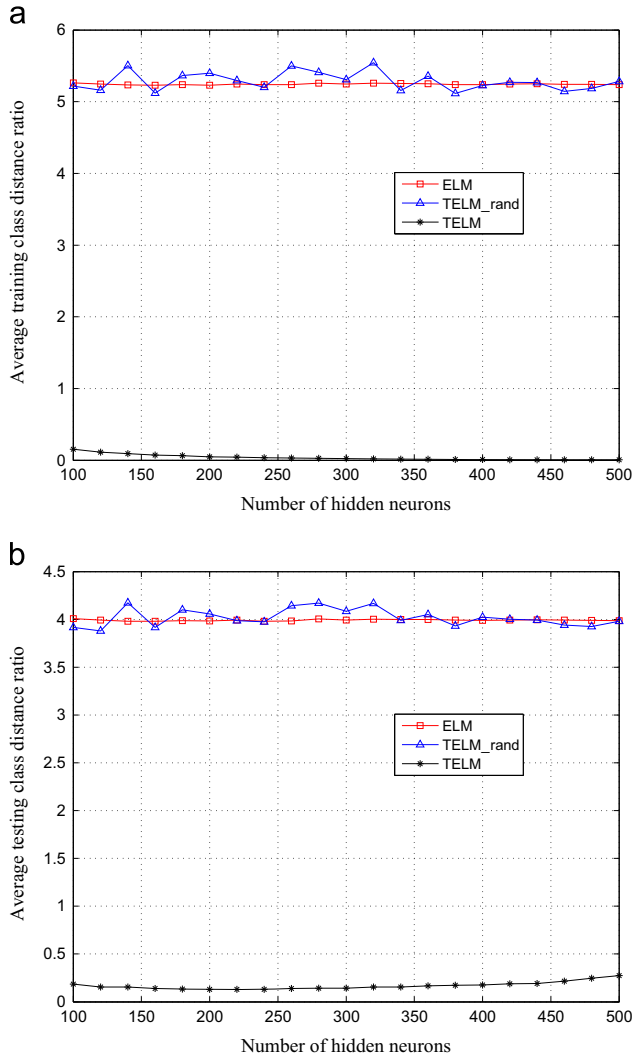


Fig. 6. Average class distance ratio for the algorithms TELM, ELM, and TELM_rand using the vowel datasets. (a) Case of the average D_{avg} of training data, and (b) Case of the average D_{avg} of testing data.

outperforms the other algorithms in terms of average error percentage.

4.3. MNIST dataset

In this section we use a more complicated classification problem, MNIST dataset [37], to further test the capabilities of the TELM algorithm. It is known that MNIST is a good dataset to test the performance of multilayer perceptrons, and that the ML-ELM method, as mentioned in [26], has shown excellent average testing accuracy on this dataset. In ML-ELM, ELM-based auto-encoders are used as basic elements in the multilayer structure. That is to say, previous hidden layers of ML-ELM are actually viewed as an efficient feature extractor for the input data. Inspired by this idea, the feature extraction scheme of ML-ELM is incorporated into the original TELM. In this manner, an improved version of TELM, called TELM_MLELM, is proposed. Specifically, in TELM_MLELM the parameters for the first hidden layer (connection weights between the input weights and the first hidden layer and the bias of the first hidden layer) are obtained by the ELM-based auto-encoder method. The parameters are simultaneously required to fulfill the

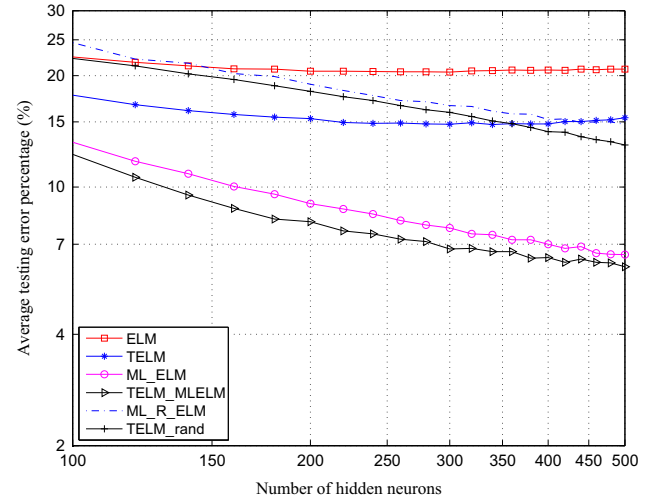


Fig. 7. Average testing classification error percentage for the algorithms TELM, ELM, ML_ELM, TELM_MLELM, ML_R_ELM, and TELM_rand using MNIST dataset.

following constraints:

$$h_{ij} = g(\mathbf{W}_j \mathbf{x}_i + b_j) \quad (i = 1, 2, \dots, N, j = 1, 2, \dots, L)$$

$$\mathbf{W}_j^T \mathbf{W}_j = \mathbf{I}, b_j^T b_j = 1$$

where $\mathbf{W}_j$ is the orthogonal random connection weights between n input neurons and the j th hidden neuron, b_j is the orthogonal random bias of the j th hidden neuron of the first hidden layer, and the parameters for the latter hidden layer (including the connection weights between the first and second hidden layer and the bias of the second hidden layer) are still calculated by the novel calculating method proposed in this paper, as done for the case of the original TELM approach.

An experiment using the MNIST dataset is conducted to assess the advantages of the proposed TELM_MLELM framework for parameter setting. Fig. 7 shows the results, plotted using logarithmic axes. From these results it is readily concluded that the TELM_MLELM algorithm has the best average testing error percentage relative to the original ELM and relative to all of the following multilayer ELM algorithms: ML-ELM, ML_R_ELM (where the parameters of first hidden layer are calculated using the ELM-based autoencoder scheme while the parameters of the second hidden layer are randomly generated), TELM_rand, and the original TELM technique. The figure also shows that the performance of the TELM algorithm is slightly worse than those of ML-ELM and TELM_MLELM. In fact this observation is logically expected given that in these cases the previous hidden layers in ML-ELM are used for feature extraction. Finally, based on the above results it can be inferred that the TELM_MLELM method is able to achieve better and more robust performance relative to all the other relevant ELM variants considered. These results further demonstrate the effectiveness of adopting the new TELM approach in an appropriate contextual fashion.

5. Conclusions and future work

A novel neural network based algorithm called TELM is proposed that, by making the actual hidden layer output approach the expected hidden layer output, improves to a significant degree both the average training and testing performance. Experimental results show that for function approximation tasks the proposed algorithm remarkably decreases the average training and testing RMSE, while for the benchmark classification problems, the

average testing error percentage is distinctly lower than those of the ELM and TELM_rand techniques when using fewer hidden neurons. In addition, to further demonstrate the effectiveness of the proposed algorithm, we conducted another experiment using the MNIST dataset and have introduced an extended TELM variant namely TELM_MLELM in which ML-ELM, as a feature extractor, is combined with the original TELM. The experimental results demonstrate that TELM_MLELM has the best average testing classification error percentage among all the algorithms considered.

The proposed TELM is a particularly attractive option for solving complex regression and classification problems in the presence of limited computational storage resources. In particular, the TELM approach can bring about significant performance advantages in applications where the number of possible hidden neurons that can be specified is limited by factors such as hardware limitations. For example, applications deployed on portable hardware are often restricted in storage as a consequence of the need to reduce product costs. In such cases, and in other analogous real-life applications, the TELM approach is able to deliver improved accuracy relative to conventional alternatives.

Future work should address an over-fitting problem observed when applying the TELM algorithm to classification tasks, and include in the scope of the study the design of an adaptive strategy to adjust the number of neurons in the second hidden layer. Additionally, in this study the maximum number of hidden neurons is limited to 500, because the research focus is placed on those practical cases where hardware limitations prevent the implementation of a large number of neurons. A question that now becomes of relevance is how the TELM approach performs when the hardware platform can afford the use of a significantly increased number of hidden neurons. The issue holds intrinsic merit, as it has been shown that the performance of SLFM extreme learning machine methods applied to the MNIST dataset can achieve accuracies close to 99% when the number of hidden nodes is as high as 5,000–10,000 [38]. The authors propose that this question is considered as a topic for future work.

Acknowledgments

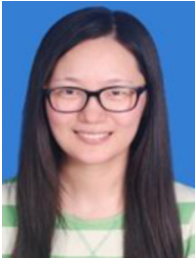
This research is partially supported by National Natural Science Foundation of China (61305080, 61473266, 61379113), the Post-doctoral Science Foundation of China (2014M552013), and the Scientific and Technological Project of Henan Province (132102210521, 152102210153).

References

- [1] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366.
- [2] K. Hornik, Approximation capabilities of multilayer feedforward networks, *Neural Netw.* 4 (2) (1991) 251–257.
- [3] G.B. Huang, Q.Y. Zhu, C.K. Siew, Extreme learning machine: theory and applications, *Neurocomputing* 70 (1) (2006) 489–501.
- [4] G.B. Huang, Q.Y. Zhu, C.K. Siew, Extreme learning machine: a new learning scheme of feedforward neural networks, In: *Proceedings of the 2004 IEEE International Joint Conference on Neural Networks*, 2004, 2, pp. 985–990.
- [5] J.M. Ortega, *Matrix Theory*, Plenum Press, 1987.
- [6] A.A. Mohammed, R. Minhas, Q.M.J. Wu, et al., Human face recognition based on multidimensional PCA and extreme learning machine, *Pattern Recognit.* 44 (10) (2011) 2588–2597.
- [7] W. Zong, G.B. Huang, Face recognition based on extreme learning machine, *Neurocomputing* 74 (16) (2011) 2541–2551.
- [8] J. Cao, Y. Zhao, X. Lai, et al., Landmark recognition with sparse representation classification and extreme learning machine, *J. Frankl. Inst.* 352 (10) (2015) 4528–4545.
- [9] J. Cao, Z. Lin, Extreme learning machines on high dimensional and large data applications: a survey, *Math. Probl. Eng.* 501 (2015) 103796.
- [10] J. Cao, T. Chen, J. Fan, Landmark recognition with compact BoW histogram and ensemble ELM, *Multimed. Tools Appl.* (2015) 1–19. [10.1007/s11042-014-2424-1](https://doi.org/10.1007/s11042-014-2424-1).
- [11] S. Cheng, J. Yan, D. Zhao, et al., Short-term load forecasting method based on ensemble improved extreme learning machine, *J. Xi'an Jiaotong Univ.* 2 (2009) 029.
- [12] L. Mao, Y. Wang, X. Liu, et al., Short-term power load forecasting method based on improved extreme learning machine, *Power Syst. Prot. Control* 40 (20) (2012) 140–144.
- [13] J. Wang, J. Hu, K. Ma, et al., A self-adaptive hybrid approach for wind speed forecasting, *Renew. Energy* 78 (2015) 374–385.
- [14] G. Wang, Y. Zhao, D. Wang, A protein secondary structure prediction framework based on the extreme learning machine, *Neurocomputing* 72 (1) (2008) 262–268.
- [15] J. Cao, Z. Lin, G.B. Huang, et al., Voting based extreme learning machine, *Inf. Sci.* 185 (1) (2012) 66–77.
- [16] W. Deng, Q. Zheng, L. Chen, Regularized extreme learning machine, In: *Proceedings of the IEEE Symposium on Computational Intelligence and Data Mining, CIDM'09*, 2009, pp. 389–395.
- [17] W.Y. Deng, Q.H. Zheng, L. Chen, et al., Research on extreme learning of neural networks, *Chin. J. Comput.* 33 (2) (2010) 279–287.
- [18] Q.Y. Zhu, A.K. Qin, P.N. Suganthan, et al., Evolutionary extreme learning machine, *Pattern Recognit.* 38 (10) (2005) 1759–1763.
- [19] N.Y. Liang, G.B. Huang, P. Saratchandran, et al., A fast and accurate online sequential learning algorithm for feedforward networks, *IEEE Trans. Neural Netw.* 17 (6) (2006) 1411–1423.
- [20] M.B. Li, G.B. Huang, P. Saratchandran, et al., Fully complex extreme learning machine, *Neurocomputing* 68 (2005) 306–314.
- [21] Z. Bai, G.B. Huang, D. Wang, et al., Sparse extreme learning machine for classification, *IEEE Trans. Cybernetics* 44 (10) (2014) 1858–1870.
- [22] G.B. Huang, H. Zhou, X. Ding, et al., Extreme learning machine for regression and multiclass classification, *IEEE Trans. Syst. Man Cybern. Part B: Cybern.* 42 (2) (2012) 513–529.
- [23] H.J. Rong, Y.S. Ong, A.H. Tan, et al., A fast pruned-extreme learning machine for classification problem, *Neurocomputing* 72 (1) (2008) 359–366.
- [24] G.B. Huang, Learning capability and storage capacity of two-hidden-layer feedforward networks, *IEEE Trans. Neural Netw.* 14 (2) (2003) 274–281.
- [25] H.-G. Han, L.-D. Wang, J.-F. Qiao, Hierarchical extreme learning machine for feedforward neural network, *Neurocomputing* 128 (2014) 128–135.
- [26] L.L.C. Kasun, H. Zhou, G.-B. Huang, C.M. Vong, Representational learning with extreme learning machine for big data, *IEEE Intell. Syst.* 28 (6) (2013) 31–34, December.
- [27] Chenwei Jiexiong, Tang Deng, Guang-Bin Huang, Extreme learning machine for multilayer perceptron, *IEEE Trans. Neural Netw. Learn. Syst.* (2015) in press.
- [28] G.B. Huang, L. Chen, Convex incremental extreme learning machine, *Neurocomputing* 70 (16) (2007) 3056–3062.
- [29] G.B. Huang, L. Chen, Enhanced random search based incremental extreme learning machine, *Neurocomputing* 71 (16) (2008) 3460–3468.
- [30] G.B. Huang, An insight into extreme learning machines: random neurons, random features and kernels, *Cogn. Comput.* 6 (3) (2014) 376–390.
- [31] D.E. Rumelhart, Learning representations by back-propagation errors, *Nature* 323 (1986) 533–536.
- [32] M.T. Hagan, M.B. Menhaj, Training feedforward networks with the Marquardt algorithm, *IEEE Trans. Neural Netw.* 5 (6) (1994) 989–993.
- [33] P.L. Bartlett, The sample complexity of pattern classification with neural networks: the size of the weights is more important than the size of the network, *IEEE Trans. Inf. Theory* 44 (2) (1998) 525–536.
- [34] S. Tamura, M. Tateishi, Capabilities of a four-layered feedforward neural network: four layers versus three, *IEEE Trans. Neural Netw.* 8 (2) (1997) 251–255.
- [35] V.L. Huang, P.N. Suganthan, J.J. Liang, Comprehensive learning particle swarm optimizer for solving multiobjective optimization problems, *Int. J. Intell. Syst.* 21 (2) (2006) 209–226.
- [36] University of California, Irvine, Machine Learning Repository. (<http://archive.ics.uci.edu/ml/>).
- [37] The Mixed National Institute of the standards and Technology (MNIST) handwriting dataset. (<http://yann.lecun.com/exdb/mnist/>).
- [38] M.D. McDonnell, M.D. Tissera, T. Vladusich, A. van Schaik, J. Tapson, Fast, simple and accurate handwritten digit classification by training shallow neural network classifiers with the 'extreme learning machine' algorithm, *PLoS One* 10 (8) (2015), Article number e0134254.



B.Y. Qu received the B.E. degree and Ph.D. degree from the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. He is an Associate Professor in the School of Electric and Information Engineering, Zhongyuan University of Technology, China. His research interests include machine learning, neural network, genetic and evolutionary algorithms, swarm intelligence, and multi-objective optimization.



B.F. Lang obtained the B.S. degree in Automation from School of Electronics and Automation, City Institute, Dalian University of Technology, Liaoning China, in 2013. She is currently pursuing her M.S degree in School of Electrical Engineering, Zhengzhou University. Her current research interests focus on Extreme Learning Machine, Pattern Recognition.



Oscar D. Crisalle is a *Distinguished Teaching Scholar and Professor* in the Chemical Engineering Department at the University of Florida and Professor at Zhengzhou University. He received a B.S. degree from the University of California, Berkeley in 1982, an M.S. degree from Northwestern University in 1986, and a Ph.D. degree from the University of California at Santa Barbara in 1990, each in chemical engineering. His current research interests focus on model-based multivariable control and instrumentation design, with applications to fuel cells and smart grid architectures. Dr. Crisalle has received numerous distinctions for his teaching, including the *2002 University of Florida Teacher of the*

Year award.



J.J. Liang received the B. Eng. degree from Harbin Institute of Technology, China and the Ph.D. degree from the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. She is currently a Professor in the School of Electrical Engineering, Zhengzhou University, China. Her main research interests are machine learning, data mining, evolutionary computation. Dr. Liang won the *2014 IEEE Computational Intelligence Society Outstanding Ph.D. dissertation award*.



A.K. Qin received the Ph.D. degree from the Nanyang Technological University (Singapore) in 2007. From 2007 to 2012, he had worked first at the University of Waterloo (Canada) and then at the French National Institute for Research in Computer Science and Control (INRIA) (France). He is now a lecturer at the RMIT University (Australia). His major research interests include evolutionary computation, machine learning, image processing, GPU computing, and service computing. He won the *2012 IEEE Transactions on Evolutionary Computation Outstanding Paper Award* and the *Overall Best Paper Award at the 18th Asia Pacific Symposium on Intelligent and Evolutionary Systems (IES*

2014). One of his conference papers was nominated for the best paper award at the *2012 Genetic and Evolutionary Computation Conference (GECCO 2012)*. Dr. Qin is an IEEE senior member, currently chairing the IEEE Emergent Technologies Task Force on "Collaborative Learning and Optimization".